PROGRAMMING WITH PYTHON II

Sessions 1 & 2

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Index

1. IDE installation VS Code
2. Virtual environments (venv)
3. First Class Objects and Decorators
4. Comprehensions and generators
5. Type Hinting and Static Analysis
6. Asynchronous I/O

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

First Steps & requirements: IDE Installation

Visual Studio Code +

https://code.visualstudio.com/download

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Virtual environments

venv

- Since Python 3.4, pip and the venv module have been available by default along with the Python installation.
- venv helps us to create isolated versions of Python environment in the same system.
- This avoids dependency conflicts (dependency hell) when working on multiple projects that rely on different versions of the same packages.
- Creation of a new environment by using the command:

```
$ python3 -m venv myproject
```

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Virtual environments

- Activate a virtual environment

```
$ source /bin/activate myproyect
```

- Deactivate a virtual environment

```
(myproject)$ deactivate
```

- **Reproducibility**
  - Save all of my explicit package dependencies into a file (which, by convention, is named requirements.txt)

```
(project)$ python3 -m freeze
```

  - I can install all of the packages from the first environment by:

```
(otherproject)$ python3 -m pip install -r /tmp/myproject/ requirements.txt
```

First Class Objects

- Functions in Python are first-class objects. Programming language researchers define a “first-class object” as a program entity that can be:
  - ➔ Created at runtime
  - ➔ Assigned to a variable or element in a data structure
  - ➔ Passed as an argument to a function
  - ➔ Returned as the result of a function
- Functions are actual instances of the function class
- The ability to treat functions as objects enables higher-order functions (H.O.F.), such as ***sorted()***, which accepts a function via the key argument

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Decorators

- A decorator is a callable that accepts another function as an argument
- They are executed immediately when the module is loaded (at import time), not when the decorated function is run (at runtime)
- **Best Practice: The @wraps Fix**
  - Decorators usually replace the original function, masking its name and docstring.
  - Use the @functools.wraps decorator when defining a new decorator to copy essential attributes like __name__ and __doc__ from the decorated function to the wrapper

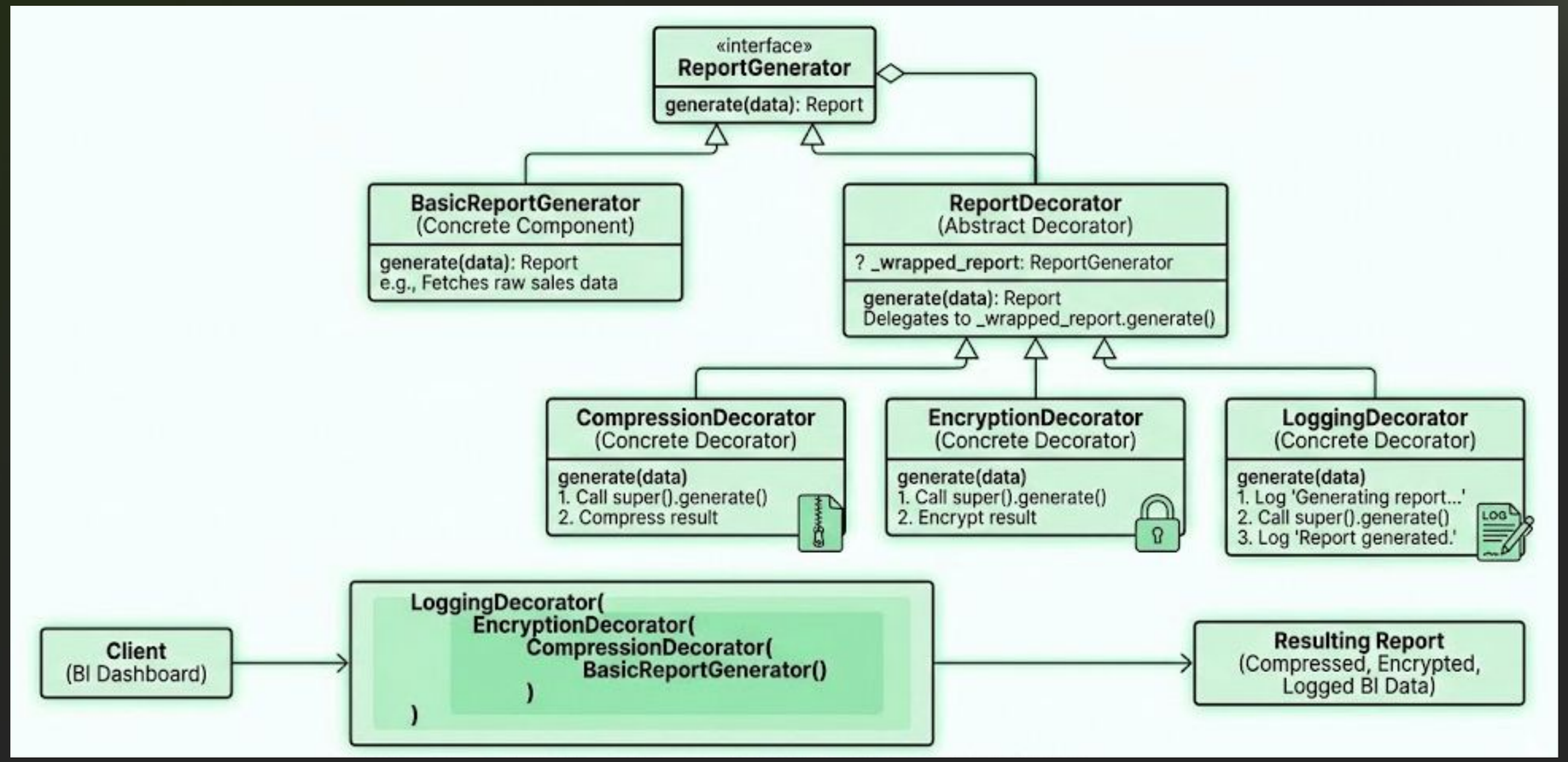
@functools.wraps

```python
import time
import functools

def clock(func):
    @functools.wraps(func)  # Preserves metadata of the original function
    def clocked(*args, **kwargs): # Added **kwargs to handle all argument types
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - t0
        name = func.__name__
        arg_str = ', '.join(repr(arg) for arg in args)
        print(f'[{elapsed:0.8f}s] {name}({arg_str}) -> {result!r}')
        return result
    return clocked
```

- Without ***@wraps***, the decorated function loses its original identity (e.g., my_function.__name__ would report as clocked instead of my_function).

Decorators: Example



ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Comprehensions

- Special syntax for succinctly iterating through these Lists, Dictionaries and Sets, and creating derivative data structures.
- Comprehensions can significantly increase the readability of code performing these common tasks and provide a number of other benefits.

<u>Syntax of list comprehensions</u>

➔ Condition:

```
newlist = [expression for item in iterable if condition == True]
```

➔ Iterable:

```
newlist = [x for x in range(10)]
```

➔ Expression:

```
newlist = [x.upper() for x in fruits]
```

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Comprehensions

Example: Use comprehensions instead of a map and filter

Say that I want to compute the square of each number in a list. Here, I do this by using a simple for loop:

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = [] for x in a: squares.append(x**2)
```

With a list comprehension, I can achieve the same outcome by specifying the expression for my computation along with the input sequence to loop over:

```
squares = [x**2 for x in a] # List comprehension
```

Unless you're applying a single-argument function, list comprehensions are also clearer than the map built-in function for simple cases. map requires the creation of a lambda function for the computation, which is visually noisy:

```
alt = map(lambda x: x ** 2, a)
```

**Dictionaries and sets have their own equivalents of list comprehensions (called dictionary comprehensions and set comprehensions, respectively). These make it easy to create other types of derivative data structures when writing algorithms

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Generators

- Generator expressions use the same syntax as list comprehensions but return a generator object (an iterator) instead of a fully realized list
- Generators are lazy: they produce outputs one at a time, only as requested
- Why should I use generators?
  - ➔ Generators allow handling arbitrarily large inputs without consuming too much working memory
  - ➔ They avoid the risk of memory explosion or crashing the program when processing large data streams
  - ➔ Generators are crucial for efficiency when processing large results or handling data streams in backend APIs.

Generators: examples

- The **yield** keyword is what makes a function a generator.

```python
def large_sequence(n):
    for i in range(n):
        yield i

# This doesn't create a million numbers in memory
gen = large_sequence(1000000)

print(next(gen))  # Output: 0
print(next(gen))  # Output: 1
print(next(gen))  # Output: 2
```

- List comprehension vs generator expression:

```python
# List comprehension - creates a list (Eager evaluation)
list_comp = [x * x for x in range(5)]
print(list_comp)

# Generator expression - creates a generator (Lazy evaluation)
gen_exp = (x * x for x in range(5))
print(gen_exp)
print(list(gen_exp))
```

When there are no more values to yield, the generator raises a `StopIteration` exception:

Type Hinting and Static Analysis

- Type hints are an optional feature in Python
- They provide syntax for declaring expected types for arguments and return values:

  name: str -> bool

- Type hints are designed for static analysis tools (like Mypy) that check code before it runs, helping to catch bugs. So there are generally ignored at runtime, as Python is dynamically typed language.
- **Mypy*** reads type hints to detect common errors, such as incompatible operations or mixing types (e.g., attempting to concatenate float and str)
- **FastAPI** leverages these type hints extensively for automatic data validation, serialization, and documentation

**We will dive deeper in these concepts and the use of Mypy in sessions 3 and 4

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Asynchronous I/0

- Python addresses the need for highly concurrent I/O with coroutines.
- Coroutines let you have a very large number of seemingly simultaneous functions in your Python programs.
- They’re implemented using the async and await keywords along with the same infrastructure that powers generators
- Coroutines run in a single thread under the supervision of the event loop. This execution model allows for highly concurrent I/O
- Crucially, the I/O functions yield control instead of blocking the entire thread, preventing the application from freezing while waiting for network responses or disk access

Asynchronous I/0

```python
import asyncio
import time

async def say_after(delay, what):
    await asyncio.sleep(delay)
    print(what)

async def main():
    print(f"started at {time.strftime('%X')}")

    await say_after(1, 'hello')
    await say_after(2, 'world')

    print(f"finished at {time.strftime('%X')}")

asyncio.run(main())
```

More info: https://docs.python.org/3/library/asyncio-task.html

Lets practice

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Bibliography

- Fluent python (Luciano Ramalho)
  - Chapter 3: Functions
  - Chapter 6: Object References, Mutability, and Recycling
  - Chapter 7: Functions as First-Class Objects
  - Chapter 9: Decorators and Closures
- Effective communication (Brett Slatkin)
  - Chapters 3 and 4: Comprehensions and Generators
  - Chapter 5: Type hinting
  - Chapter 7: Asynchronous I/O
  - Chapter 10: Virtual environments
- Asynchronous I/O: Python documentation
  - https://docs.python.org/3/library/asyncio-task.html

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY